Laboratório de Programação II

Valores, Tipos de Dados e Operadores

Prof. Esp. Leonardo César Bottaro

Tipagem Dinamica

- JavaScript e uma linguagem de tipagem dinamica.
  Uma variavel pode armazenar qualquer tipo de valor, e o tipo pode mudar durante a execucao.
- Exemplo:

```
let x = 10;  // x é number
x = "texto"; // x agora é string
x = true;    // x agora é boolean
```

Classificação dos Tipos

Categoria	Descrição
Primitivo	Valor imutável, copiado por valor.
	undefined, null, boolean, number, string, symbol, bigint
Objeto	Valor mutável, copiado por referência.
	Object (inclui Array, Function, Date, RegExp, Map, Set etc.)

Undefined

Representa a ausência intencional de valor.

- Variável declarada sem inicialização tem valor undefined.
- Função sem return retorna undefined.

```
let a;
console.log(a); // undefined
```

Null

- Representa a **ausência deliberada** de um objeto.
  Usado para indicar “nenhum objeto”.
- ⚠ **Atenção:** typeof null retorna "object" → um bug histórico.

```
let pessoa = null;
```

Boolean

- Apenas dois valores: true e false.
  Utilizado em condições e operações lógicas.

```
let ativo = true;
let concluido = false;
```

Number

- Representa **todos** os números (inteiros e ponto flutuante).
- Padrão IEEE 754 (dupla precisão)
- Valores especiais: Infinity, -Infinity, NaN.

```
let inteiro = 42;
let decimal = 3.14;
console.log(1 / 0);          // Infinity
console.log(0 / 0);          // NaN
console.log(Number.MAX_SAFE_INTEGER); // 9007199254740991
```

String

- Representa texto.
- **Imutável***: operações criam novas strings.
- Delimitadores: aspas simples, duplas ou *backticks* (template literals).

```
let nome = "Léo";
let sobrenome = 'Bottaro';
let mensagem = `Olá, ${nome} ${sobrenome}!`;
console.log(mensagem); // "Olá, Léo Bottaro!"
```

Symbol (ES6) e BigInt (ES2020)

- **Symbol**: valor único e imutável. Usado para chaves de propriedades.
- **BigInt**: inteiros maiores que 2^53 - 1. Sufixo n.

```
let s1 = Symbol("id");
let s2 = Symbol("id");
console.log(s1 === s2); // false

let grande = 9007199254740991n;
let outro = BigInt("123456789012345678901");
```

Objetos (Object)

- Coleção de propriedades (chave → valor).
- Sub-tipos: Array, Function, Date, RegExp etc.

```
let pessoa = { nome: "João", idade: 30 };
let frutas = ["maçã", "banana"];
function soma(a, b) { return a + b; }

console.log(typeof pessoa); // "object"
console.log(typeof frutas); // "object"
console.log(typeof soma);   // "function"
```

Valor vs Referência

- **Primitivos**: copiados e comparados por valor.
- **Objetos**: copiados e comparados por referência.

```
// Primitivo – por valor
let a = 10;
let b = a;
b = 20;
console.log(a); // 10 (não mudou)

// Objeto – por referência
let obj1 = { valor: 10 };
let obj2 = obj1;
obj2.valor = 20;
console.log(obj1.valor); // 20
```

Operadores Aritméticos

+ (adição/concatenação)
- (subtração)
* (multiplicação)
/ (divisão)
% (modulo, resto da divisão)
** (exponenciação)
++ (incremento)
-- (decremento)

```
let x = 10;
console.log(x + 5);   // 15
console.log(x % 3);   // 1
console.log(x ** 2);  // 100
console.log(++x);     // 11 (pré-incremento)
```

Operadores de Atribuição

= atribuição
+= soma e atribui
-= subtrai e atribui
*=
/=
%=
**=

```
let total = 100;
total += 50;  // 150
total *= 2;   // 300
total /= 3;   // 100
```

Operadores de Comparação

== (igualdade frouxa, com coerção)
=== (igualdade estrita, sem coerção)
!= (diferença frouxa)
!== (diferença estrita)
> (maior que)
< (menor que)
>= (maior ou igual a)
<= (menor ou igual a)

```
console.log(5 == "5");   // true
console.log(5 === "5");  // false
console.log(0 == false); // true
console.log(0 === false);// false
```

Operadores Lógicos

&& (E lógico / AND)
|| (OU lógico / OR)
! (NÃO lógico / NOT)

Curto-circuito:

- Em &&, se o primeiro for false, o segundo não é avaliado.
- Em ||, se o primeiro for true, o segundo não é avaliado.

```
let idade = 20;
console.log(idade !== undefined && idade >= 18); // true
let nome = "";
console.log(nome || "Visitante"); // "Visitante"
```

Operador Ternário

- Sintaxe:
  condição ? valor_se_verdade : valor_se_falso

```
let idade = 18;
let status = idade >= 18 ? "Adulto" : "Menor de idade";
console.log(status); // "Adulto"
```

Outros Operadores

- typeof → retorna o tipo do operando.
- delete → remove propriedade de um objeto.
- in → verifica se a propriedade existe.
- instanceof → verifica se é instância de um construtor.
- void → retorna undefined.
- Operador vírgula , → avalia vários e retorna o último.

```
console.log(typeof "texto");              // "string"
console.log("marca" in { marca: "Fiat" }); // true
console.log([] instanceof Array);         // true
```

Conversão Explícita (Type Casting)

- String(valor) → converte para string.
- Number(valor) → converte para número.
- Boolean(valor) → converte para booleano.
- parseInt() / parseFloat() → convertem parcialmente.

```
console.log(String(123));         // "123"
console.log(Number("456"));       // 456
console.log(Boolean(0));          // false
console.log(parseInt("123px"));   // 123
```

Coerção de Tipos (Implícita)

- JavaScript converte tipos automaticamente em operações.

```
console.log("5" + 2);   // "52" (número vira string)
console.log("5" - 2);   // 3   (string vira número)
console.log("10" / 2);  // 5
console.log("abc" - 1); // NaN



if ("")
        console.log("não executa");  // string vazia → falsy

if ("texto")
        console.log("executa"); // string não vazia → truthy
```

Truthy e Falsy

- **Falsy** (avaliam para false):
  false, 0, -0, 0n, "", null, undefined, NaN
- **Truthy** (avaliam para true):
  todos os demais valores, incluindo [], {}, " ", Infinity.

```
if ([]) console.log("array vazio é truthy");
if ({}) console.log("objeto vazio é truthy");
if (null) console.log("não executa");
```

Gotchas: Armadilhas Comuns

- typeof null → "object". Use === null para verificar
- NaN === NaN → false. Use isNaN() ou Number.isNaN()
- 0 == false → true, mas 0 === false → false. Prefira ===
- "5" + 2 → "52", mas "5" - 2 → 3. Cuidado com +
- null > 0 → false, null == 0 → false, mas null >= 0 → true

Gotchas: Armadilhas Comuns

- "2" > "12" → true (comparação lexicográfica).
- 0.1 + 0.2 === 0.3 → false (precisão IEEE 754).
- [] + [] → "" (string vazia).
- [] + {} → "[object Object]".
- {} + [] → 0 (em alguns contextos).
- typeof NaN → "number".

Exercício Prático

1. Qual o resultado de typeof null?
2. Qual o resultado de 10 + "20"?
3. Qual o resultado de 10 - "20"?
4. Qual o resultado de 0 == false?
5. Qual o resultado de 0 === false?
6. Qual o resultado de !!"" (dupla negação)?
7. Qual o resultado de !!"texto"?
8. Crie um objeto pessoa com nome e idade. Verifique se a propriedade "nome" existe usando o operador in.
9. Converta a string "3.1415" para número usando Number e parseFloat.
10. Verifique se NaN é igual a NaN.